VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

# BASIC PROGRAMMING

Lecture 6: Pointers

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong

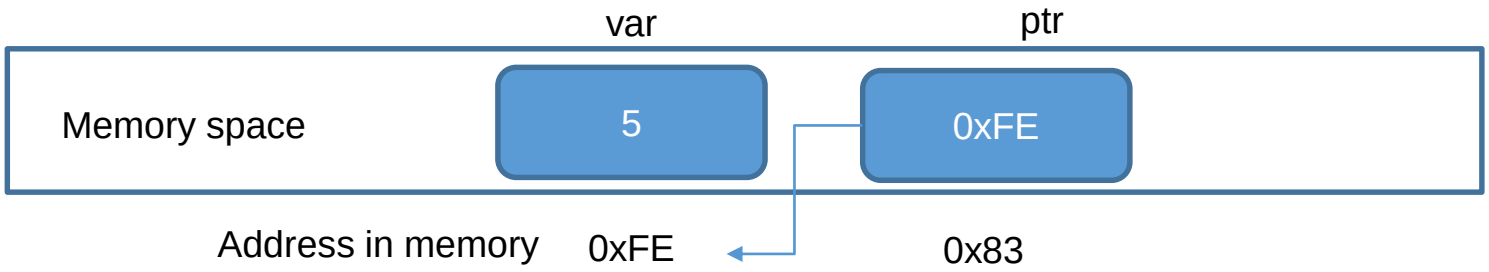
# Lecture 6: Pointer

- Introduction to Pointers
- Operations with Pointers
- Pointers and Arrays
- Manage Dynamic Memory using Pointers

# Introduction

# Introduction

- What is pointer?
  - A variable contains the address in memory of another variable




```
int var = 5;
int* ptr = &var;
printf("And the value pointed by ptr: %d\n", *ptr);
```

  - operator & : address of a variable
  - operator * : contents of the variable pointed by the pointer

# Introduction (cont')

- Type of data stored in the variable pointed by a pointer must be specific
- Pointer is used only for pointing to a variable

# Pointer declaration

- Syntax:

  Type * pointer name;

- Examples:

```
int* i_pointer;
float* f_pointer;
double* d_pointer;
```

# Pointer initialization

- Initialize a pointer when you declare it

```
int* i_pointer = &number;
```

Address of
variable “number”

- Pointer to NULL: refers to nowhere in memory

```
int* i_pointer = NULL;//i_pointer is now 0
```

# Accessing a value through a pointer

- To access the value of the variable pointed by the pointer:
  - Operator *
  - Example:

```
int number = 18;
int* i_pointer = &number;
int result = *i_pointer + 5;
```

# Passing and returning pointers to/from functions

- Passing pointers to functions:

  Declare function parameters as pointer type

Function declaration

```
void greeting (char* a)
{
        printf(" %s \n", a);
}
```

Function call

```
int main()
{
    // // array declaration
    char B[] = {"How are you?"};
    greeting(B);
    return 0;
}
```

# Passing and returning pointers to/from functions

- Return pointers from functions:

Function declaration

```
int* increase_elements(int arr[], int
size)
{
    for (int i =0;i<size;i++)
    {
        arr[i] = arr[i] + 1;
    }
    return arr;
}
```

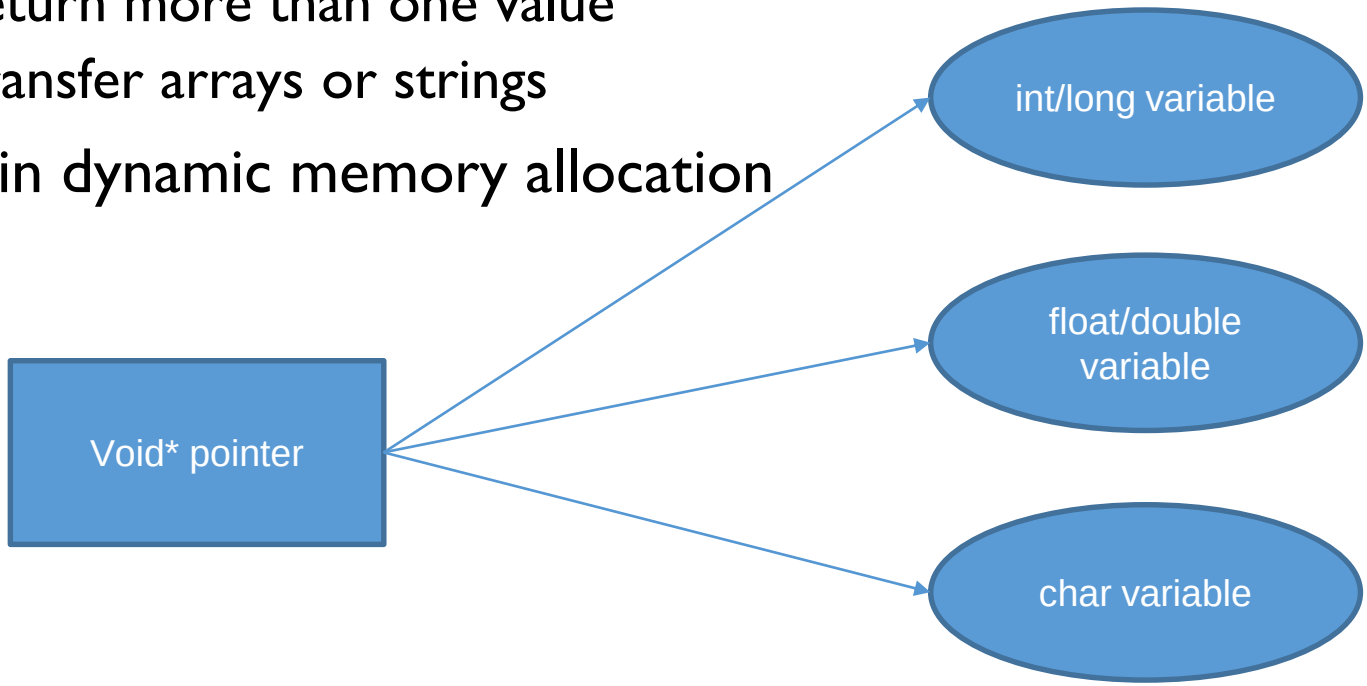
Function call

```
int main()
{
// // array declaration
          int B[] = {1,2,3,4,5};
          int *p;
          p= increase_elements(B,5);
          for (int i=0; i<5; i++)
          {
                    printf(" %d ",*p+i);
          }
          printf (" \n ");

          return 0;
}
```

# Void Pointers

- Can point to a variable of any type
- Often used with functions:
  - Return more than one value
  - Transfer arrays or strings
- Used in dynamic memory allocation



# Pointers to Constant

- Use `const` keyword when declaring a pointer: value pointed by the pointer must not be changed
- Declaration:

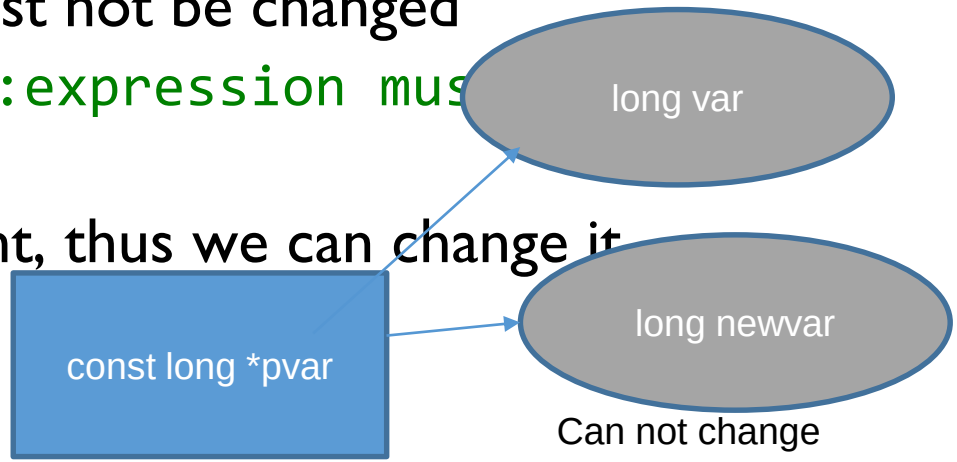
```
long var = 9999L;
const long* pvar = &var;
```

- Point to the value that must not be changed

```
*pvar = 19L; //error:expression mus
modifiable value
```

- The pointer is not constant, thus we can change it

```
long newvar = 19L;
pvar = &newvar;
```



# Constant Pointers

- The address stored in the pointer cannot be changed

```
long var = 9999L;
long *const pvar = &var;
```

- But the value pointed by a const pointer can be changed

```
int main()
{
        int var1 = 5;
        int var2 = 10;
        int* const p = &var1;//a constant pointer refering to an address all the time
        *p = 8;//posible to update the variable pointed by p

        printf("Value of p is %d, and value pointed by p is %d", p, *p);
        p = &var2;//error here since p is a constant pointer
        return 0;
}
```

# Constant Pointers to Constants

- The address stored in the pointer and pointer itself cannot be changed

```
long var = 9999L;
const long *const pvar = &var;
```

# Operations with Pointers

# Operation with Pointers

- Pointer arithmetic
  - Increasing
  - Decreasing
- Pointer comparisons

# Pointer Arithmetic

- Four arithmetic operators: ++, --, +, -
- Incrementing a pointer:

```
int main()
{
	int numbers[] = {2,4,6,8,10};
	int size = 5;

	int* p = numbers;
	for (int i =0; i<size;i++)
	{
		printf("The #%d element is %d \n", i,*p);
		p ++;//let p point to the next element.
	}
	return 0;
}
```

# Pointer with Arrays and Strings

# Pointer and 1-D Arrays

- The 1-D array name is infact a const pointer to its first element

```
int main()
{
        int A[] = {1,2,3,4,5};
        A++;//error since A is a const pointer
        return 0;
}
```

- Increase/reduce value of p to travel along the array elements

```
int main()
{
        int A[] = {1,2,3,4,5};
        int* p = A;//p points to A[0]
        p++;//p points to A[1]
        printf("The next 3 element is %d \n", p[3]);/*p points to A[4] so p[3] now is 5*/
        A++;//error because A is a constant pointer
        return 0;
}
```

# Pointer and String

- A pointer to char can be considered as a string
- Two arrays of char can not be assigned to each other, but two pointers to char can

```
char *t = "This is an array of char";
char *s = t;/*equivalent to strcpy(s,t), s and t have same value and point to the same address*/
```

# Manage Dynamic Memory using Pointers

# Using Memory as You Go

- How to create an array of intergers (e.g: for students' score) with size determined at runtime?
- → Dynamic memory allocation: variables are stored at memory area called heap (not stack for static variables)



# Allocate Dynamic Memory

- malloc(size) in <stdlib.h> library: standard function to allocate memory
  - Specify number of bytes you want to allocate
  - malloc() return the address of first bytes allocated

```
int *ptr = (int*) malloc(10 * sizeof(int));
```

  - If memory can not be allocated, malloc() returns NULL → checking value returned by malloc() is a good practice

```
if(ptr==NULL)
        printf("Memory can not be allocated\n");
```

- calloc(num,size) in <stdlib.h> library: similar to malloc() except it initializes num memory blocks with zero

# Release Dynamic Memory

- You should always release memory when it is no longer used to prevent memory leaking
- Memory in stack (for normal variables) is released when the scope of variables is over
- Memory in heap is released automatically when the program ends → sometimes too late → better to release when you do not use it any more
- free(pointer_to_memory) function is the standard way to release memory

# Example of using malloc() and free()

```
#include <stdio.h>
#include <stdlib.h>

//using dynamic memory, compute the sum of n integers entered by user
int main()
{
	int n, i, *ptr, sum = 0;

	printf("Enter number of elements: ");
	scanf("%d", &n);

	ptr = (int*) malloc(n * sizeof(int));//do not declare fixed size array anymore
	// if memory cannot be allocated
	if(ptr == NULL){
		printf("Error! memory not allocated.");
		exit(0);
	}

	printf("Enter elements: ");
	for(i = 0; i < n; ++i)
	{
		scanf("%d", ptr + i);
		sum += *(ptr + i);
	}
	printf("Sum = %d", sum);
	// deallocating the memory
	free(ptr);

	return 0;
}
```